14 · Streaming and Real-Time ML

> **Goal:** After this chapter you can decide whether a use case really needs streaming, compute windowed features correctly under late and out-of-order data, keep online and offline features in agreement, update models incrementally with guardrails, and operate a real-time scoring pipeline with backpressure and dead letters.

Table of Contents

- 14.1 Real-Time ML Architectures: When Streaming Is Worth It
- 14.2 Streaming Feature Computation: Windows, Watermarks and Late Data
- 14.3 Online/Offline Parity and Backfilling Streaming Features
- 14.4 Online Learning and Incremental Model Updates
- 14.5 Real-Time Inference Pipelines: Stateful Scoring, Backpressure and Dead Letters

14.1 Real-Time ML Architectures: When Streaming Is Worth It

Conceptual Deep Dive

Treat architecture as a **decision ladder** and climb only as far as the value of freshness justifies. Rung 1: **batch precompute** (daily or hourly scores in a table). Rung 2: **request-time features**, computed from the request payload or a cheap lookup. Rung 3: **near-real-time micro-batch** (minutes). Rung 4: **true streaming**, with per-event state and seconds-level freshness. Each rung adds always-on infrastructure, on-call load and failure modes, so write down what an extra minute of staleness costs in business terms before paying for it. Rule of thumb: an hourly micro-batch satisfies a large majority of "real-time" requests.

Lambda architecture runs a batch and a speed layer in parallel and merges results, which means two code paths that must agree. **Kappa** uses one streaming path and reprocesses by replaying the log. Unified engines (Flink, Spark Structured Streaming) let one definition run in batch and streaming modes, which shrinks, but does not remove, the parity problem (see 14.3).

Components: an event log (Kafka-class), a stream processor (Flink, Spark Structured Streaming, Kafka Streams), an online store for low-latency reads, a model service, and a prediction log that feeds monitoring and training. Check current docs for the engine you choose; semantics differ in state backends and exactly-once support.

Freshness SLOs have three parts: event-to-feature latency, feature-to-score latency and end-to-end decision latency. Measure each with timestamps attached at the producer, at the feature write and at the score, and alert on percentiles, not means.

Cost model: streaming is always-on, scales with peak rather than average, and needs state management and replay tooling; micro-batch pays only while running. Typical streaming-worthy cases: fraud velocity features, session-based recommendation, dynamic pricing, anomaly detection and ETA estimation.

Edge cases: **bursts** create consumer lag that raises freshness latency exactly when it matters; **hot keys** skew a partition, so one consumer is saturated while others idle; **reprocessing after a bug** replays history at full speed and can flood downstream sinks, so throttle replays and write them to separate outputs.

Cross-Connection Scenarios

- Kafka scoring consumer example: Module 06 section 6 (legacy).
- Online and offline stores: Module 02 section 2 (legacy).
- Windows, watermarks and late data: 14.2; parity and replay: 14.3.
- Lag-based autoscaling with KEDA: Module 09 section 3 (legacy) and 14.5.

Production Anti-Patterns

- Streaming everything because it sounds modern: you pay always-on cost and on-call load for features whose value does not change within the hour.
- Two independent implementations of the same feature (batch and stream): they diverge silently and online accuracy falls below offline.
- No replay plan: after a bug or a schema change you cannot rebuild state, so the fix is a manual and risky data patch.

Minimal Working Code

A freshness SLI from event and feature-write timestamps: percentile lags and the share of events within the SLO (missing writes count as breaches).

```
import random

def freshness_report(event_ts, write_ts, slo_s):
    lags = sorted(write_ts[i] - event_ts[i] for i in write_ts if i in event_ts)
    def pct(p):
        return lags[min(len(lags) - 1, int(p * len(lags)))]
    within = sum(lag <= slo_s for lag in lags)
    return {"p50": round(pct(0.50), 2), "p95": round(pct(0.95), 2),
            "within_slo": round(within / len(event_ts), 3),
            "missing": len(event_ts) - len(lags)}

rng = random.Random(0)
events = {i: float(i) for i in range(1000)}
writes = {i: t + rng.expovariate(1 / 2.0) for i, t in events.items() if rng.random() > 0.02}
print(freshness_report(events, writes, slo_s=5.0))
```

The ladder as a testable function: freshness requirement, load and team size map to a recommended architecture with reasons.

```
def recommend(freshness_s, uses_event_history, streaming_engineers, qp
s=0):
    why = []
    if freshness_s >= 6 * 3600:
        why.append("staleness of hours is acceptable")
        return "batch precompute", why
    if not uses_event_history:
        why.append("features depend only on the request payload or a cheap lookup")
        return "request-time features", why
    if freshness_s >= 15 * 60:
        why.append("minutes of staleness is acceptable; scheduled jobs are cheaper than always-on st
ate")
        return "micro-batch", why
    if streaming_engineers < 2:
        why.append("sub-15-minute freshness with event history, but no team to run streaming")
        return "micro-batch now, or a managed streaming service", why
    why.append("tight freshness over event history and a team able to operate it")
    if qps > 10_000:
        why.append("high volume: plan partitioning and hot-key handling")
    return "true streaming", why

assert recommend(86400, True, 0)[0] == "batch precompute"
```

```
assert recommend(1, False, 0)[0] == "request-time features"
assert recommend(30, True, 3, qps=50_000)[0] == "true streaming"
print(recommend(60, True, 1))
```

Interview Angle

Q: Does this use case need streaming features? Take fraud, recommendations and churn and defend each. **A:** Fraud: yes for velocity features (counts in the last minutes), because attackers act within minutes; other features can be batch. Recommendations: session context often needs streaming or request-time features, while long-term taste is batch. Churn: no, daily batch suffices because labels and actions are slow. Trade-off: freshness gain versus always-on cost. Check: offline experiment showing metric lift when feature lag grows from seconds to an hour.

Q: How do you avoid maintaining two implementations of a feature? **A:** Define the feature once and compile it to both paths (a feature-store transformation or a unified engine), or log the served feature values and train on those, or replay raw events through the same streaming job. Trade-off: framework constraints versus flexibility. Check: a continuous parity test comparing offline recomputation with logged online values, with a per-feature mismatch tolerance.

14.2 Streaming Feature Computation: Windows, Watermarks and Late Data

Conceptual Deep Dive

Event time is when something happened; **processing time** is when the system saw it. Training-relevant features must use event time, otherwise replays and delays change values. Windows slice event time: **tumbling** (fixed, non-overlapping), **sliding** (fixed length, advancing by a smaller hop; the basis of "transactions in the last 10 minutes") and **session** (closed by a gap of inactivity). Common aggregates are counts, sums, distinct counts and recency.

A **watermark** is the processor's estimate that no events older than time W are still to come. It trades completeness against latency: a longer delay admits more stragglers but delays results. **Allowed lateness** keeps window state open past the watermark for corrections, and **side outputs** capture events later than that so they can be counted, alerted on and reconciled in batch.

State is keyed (per entity), grows with key cardinality and window length, and must have a **TTL**. It is checkpointed so a failed job resumes without loss, using embedded stores such as RocksDB-class backends. Watch checkpoint size and duration.

Delivery semantics: at-least-once processing plus **idempotent writes** (dedup by event id, upsert by key) gives effectively-once results; true exactly-once uses transactions between source, state and sink and holds only if every hop participates.

Approximations bound memory: **HyperLogLog** for distinct counts, **count-min sketch** for frequencies (never under-counts, over-counts under collisions), and **exponentially decayed counters** for recency-weighted velocity in constant space per key. **Hot keys** cause skew: salt the key and merge partial aggregates, or pre-aggregate before the shuffle.

Need	Technique	Cost
last N minutes count	sliding window by buckets	state per key per bucket
distinct users	HyperLogLog	small error, tiny state
top frequencies	count-min sketch	one-sided over-count
recency-weighted	decayed counter	no exact window

Edge cases: **out-of-order events** straddle window boundaries and must update the right window; an **idle partition** stops advancing its watermark, stalling the whole job unless idleness is configured; **unbounded key cardinality** (for example per-session IDs) grows state until the job fails.

Cross-Connection Scenarios

- Freshness and architecture choice: 14.1.
- Feature store consistency between stores: Module 02 sections 2-3 (legacy).
- Dedup and DLQ on the consumer side: 14.5.
- State size and OOM as an incident type: Module 11 (legacy).

Production Anti-Patterns

- Using processing time for training-relevant features: replays and delays change values, so training and serving disagree.
- No TTL on state: memory grows until the job crashes and recovery from a huge checkpoint takes hours.
- Assuming exactly-once end to end without idempotent sinks: one non-transactional hop produces duplicates that inflate counts.

Minimal Working Code

A sliding-window counter over event time with a watermark: it tolerates bounded out-of-order arrival, evicts old state and diverts late events to a side output.

```
from collections import defaultdict

class WindowCounter:
    def __init__(self, bucket=10, window=60, lateness=5):
        self.bucket, self.window, self.lateness = bucket, window, lateness
        self.state = defaultdict(lambda: defaultdict(int))   # key -> bucket index -> count
        self.max_ts, self.late = 0, []

    def watermark(self):
        return self.max_ts - self.lateness

    def add(self, key, ts):
        if ts < self.watermark():
            self.late.append((key, ts))          # side output: later than the watermark
            return
        self.max_ts = max(self.max_ts, ts)
        self.state[key][ts // self.bucket] += 1
        floor = (self.watermark() - self.window) // self.bucket
        for buckets in self.state.values():      # TTL: drop buckets outside the window
            for b in [b for b in buckets if b < floor]:
                del buckets[b]

    def count(self, key):
        floor = (self.watermark() - self.window) // self.bucket
        return sum(c for b, c in self.state[key].items() if b >= floor)

wc = WindowCounter()
for key, ts in [("u1", 100), ("u1", 103), ("u1", 101), ("u1", 120
), ("u1", 110), ("u1", 50)]:
    wc.add(key, ts)
print("count:", wc.count("u1"), "late side output:", wc.late)
```

An exponentially decayed counter and a count-min sketch show memory-bounded velocity features and the one-sided over-count error.

```
import hashlib, math, random
from collections import Counter

class Decayed:
    def __init__(self, half_life):
        self.rate, self.value, self.t = math.log(2) / half_life, 0.0, 0.0
    def add(self, ts, amount=1.0):
        self.value = self.value * math.exp(-self.rate * (ts - self.t)) + amount
        self.t = ts

class CountMin:
    def __init__(self, width=256, depth=4):
        self.w, self.d = width, depth
        self.rows = [[0] * width for _ in range(depth)]
    def _idx(self, key, i):
        return int(hashlib.sha256(f"{i}:{key}".encode()).hexdigest(), 16) % self.w
    def add(self, key):
        for i in range(self.d):
```

```
            self.rows[i][self._idx(key, i)] += 1
    def estimate(self, key):
        return min(self.rows[i][self._idx(key, i)] for i in range(self.d))

rng = random.Random(0)
cm, truth = CountMin(width=32), Counter()
for _ in range(5000):
    k = f"card{int(rng.paretovariate(1.2))}"
    cm.add(k); truth[k] += 1
over = [cm.estimate(k) - c for k, c in truth.items()]
assert min(over) >= 0                    # count-min never under-counts
print("keys:", len(truth), "mean over-count:", round(sum(over) / len(over), 2))
d = Decayed(half_life=60.0)
for ts in (0, 30, 60, 120):
    d.add(ts)
print("decayed velocity:", round(d.value, 3))
```

Interview Angle

Q: Explain watermarks and what happens to events that arrive after them. **A:** A watermark is the system's claim that event time has advanced to W, so windows ending before W can be finalised. Events older than the watermark are late: depending on policy they update a still-open window within allowed lateness, go to a side output for counting and batch reconciliation, or are dropped. Trade-off: longer delay gives completeness but adds latency. Check: late-event rate and the freshness percentile after changing the delay.

Q: Design velocity features (txns in last 1m/10m/1h) for fraud with memory bounds. **A:** Key by card, keep bucketed counts (for example 10-second buckets for 1m and 10m, minute buckets for 1h) with TTL, or decayed counters for constant state. Use count-min or HyperLogLog only for high-cardinality side features. Handle hot merchants by salting. Trade-off: bucket granularity versus accuracy and state. Check: state bytes per key, and error versus an exact offline recomputation on sampled cards.

14.3 Online/Offline Parity and Backfilling Streaming Features

Conceptual Deep Dive

The **parity problem**: training features are computed by batch SQL over history while serving features come from stream code with different windowing, null handling, time zones and late-data rules. The model learns one distribution and meets another, which shows up as a gap between offline and online metrics that no drift monitor on the model will explain.

Three strategies. **Single definition compiled to both** (feature-store transformations, or a unified engine) removes the second implementation. **Log-and-wait** logs the feature values actually served and trains on those, giving perfect parity at the cost of waiting for history and being unable to create features retroactively. **Replay raw events** through the same streaming job to generate training data, which reuses the code but needs retained raw events.

Point-in-time correctness for streams: join labels to features by **as-of** semantics on the time a value was *available to the serving system*, not merely its event time. A late-arriving event that was backdated to before the label time was unknown at that moment and must not leak into the training row.

Backfill by replaying the log from retention or from lakehouse snapshots, with checkpoint isolation (a backfill job never shares state or consumer group with the live job) and output to versioned tables, then validate and promote. Never write backfill output straight into the live online store.

Parity testing samples entities and times, recomputes features offline, compares with logged online values, and sets tolerances per feature (counts exact, ratios within an epsilon). Classify mismatches: late events, logic differences, time-zone or rounding bugs.

Retention limits are the hidden constraint: use log compaction where only the latest value matters, tiered storage for long retention, and archive raw events to object storage in a replayable format.

Edge cases: **log retention shorter than the backfill window** makes a two-year backfill impossible from Kafka alone, so use the archive; **replay joins current reference data** (dimension tables) instead of as-of snapshots, leaking the present into the past; **stream job logic changes mid-backfill** so history is computed by two versions.

Cross-Connection Scenarios

- Point-in-time joins in batch: Module 02 section 2 (legacy).
- Windowing and late data that cause mismatches: 14.2.
- Idempotent, versioned backfill outputs: 5.16.
- Training-serving skew as an incident pattern: Module 11 (legacy).

Production Anti-Patterns

- Trusting that two codepaths agree: skew appears only in production, and weeks pass before someone compares values.
- Backfilling into the live online store: the model serves half-recomputed values mid-job and a bug cannot be undone.
- Training on features recomputed with knowledge not available at the time: offline metrics look excellent and online performance collapses.

Minimal Working Code

An as-of join of labels to a feature-update log using bisect, keyed on availability time so a late-arriving backdated value does not leak.

```
from bisect import bisect_right
from collections import defaultdict

def build_index(updates):
    """updates: (entity, event_ts, available_ts, value); index by availability time."
"""
    grouped = defaultdict(list)
    for ent, event_ts, avail_ts, val in sorted(updates, key=lambda u: u[2]):
        grouped[ent].append((avail_ts, event_ts, val))
    return {e: ([r[0] for r in rows], rows) for e, rows in grouped.items()}

def as_of(index, entity, t):
    if entity not in index:
        return None
    keys, rows = index[entity]
    i = bisect_right(keys, t)            # last update available at or before t
    return rows[i - 1] if i else None

updates = [("u1", 90, 91, 3), ("u1", 100, 101, 5), ("u1", 95, 130, 4)]   #
last one arrives late
idx = build_index(updates)
labels = [("u1", 120), ("u1", 140)]
for ent, t in labels:
    row = as_of(idx, ent, t)
    assert row is None or row[0] <= t                    # no future availability leaks
    print(f"label at t={t}: feature value {row[2]} (event_ts {row[1]})")
```

A parity checker comparing offline-recomputed features with logged online values, attributing mismatches to late events versus logic differences.

```
import random

def parity_report(offline, online, tol, late_keys):
    mismatches = {"late_event": 0, "logic": 0}
    for key, off_val in offline.items():
        if abs(off_val - online[key]) > tol:
            mismatches["late_event" if key in late_keys else "logic"] += 1
    total = sum(mismatches.values())
    return {"checked": len(offline), "mismatch_rate": round(total / len(offline), 3),
 **mismatches}

rng = random.Random(0)
offline = {(f"u{i}", 100): float(rng.randint(0, 9)) for i in range(500)}
online = dict(offline)
late_keys = {k for k in offline if rng.random() < 0.05}      # entities with late events
for k in late_keys:
    online[k] -= 1.0                     # online missed the late event
```

```
for k in rng.sample(sorted(offline), 10):
    if k not in late_keys:
        online[k] += 2.0                # injected logic bug
print(parity_report(offline, online, tol=0.0, late_keys=late_keys))
```

Interview Angle

Q: Offline AUC 0.9, online 0.75, features come from a stream. How do you find the cause? **A:** First rule out label delay and population differences, then test feature parity: sample entities, recompute features offline as-of the logged prediction time and compare with the logged online values per feature. Rank features by mismatch rate times importance. Attribute mismatches to late events, time zones or logic. Trade-off: the investigation needs prediction logging that was in place. Check: AUC recomputed offline using the logged online features, which isolates feature skew from model issues.

Q: How do you backfill a new streaming feature for two years of history? **A:** Replay archived raw events (retention in the log is usually too short) through the same job code, with isolated checkpoints, as-of reference data and a versioned output table. Validate against a sample computed independently, then promote the table for training, and populate the online store from the latest state. Trade-off: replay cost versus a separate batch implementation that may drift. Check: parity report on a held-out sample.

14.4 Online Learning and Incremental Model Updates

Conceptual Deep Dive

Model updating is a **spectrum**: periodic full retrain (simple, reproducible), warm-start or fine-tune from the last model, mini-batch incremental updates (minutes to hours), and true **online learning** with a parameter update per event. Moving right buys faster adaptation and costs reproducibility, testability and safety.

Algorithms that support incremental updates include SGD-trained linear and logistic models, Hoeffding trees (streaming decision trees), approximations of online gradient-boosted trees, factorisation machines and **bandits** (see 7.18). Libraries such as River provide streaming estimators; check current docs for supported algorithms. Deep networks can be fine-tuned incrementally but are prone to forgetting.

Drift adaptation uses forgetting factors, windowed training on recent data, or **drift detectors** (ADWIN, Page-Hinkley) that watch an error or score stream and trigger a reset or retrain. Evaluate online learners with **prequential (test-then-train) evaluation**: predict on each example first, score it, then update, so every score is out-of-sample and the metric tracks the changing model.

Risks: catastrophic forgetting of rare but important patterns, **feedback loops** (the model shapes the data it later learns from), **poisoning** by adversarial or corrupted labels, and non-reproducibility. **Guardrails**: a shadow model that trains on the same stream but is not promoted, clipped update magnitudes, quarantine of suspicious labels, and rollback to a snapshot.

Serving updated weights: publish versioned **snapshots** every N minutes, canary a snapshot before it takes all traffic, and keep snapshots immutable so predictions can be explained later. For large models, parameter-server patterns separate training from serving replicas.

Also decide who may learn: updates from labels that a human or downstream system produced should be sampled and audited before they influence weights, and each update should carry the data version it learned from. Edge cases: a **burst of adversarial labels** shifts the model quickly, so rate-limit label influence; a **drift detector fires on seasonality** (Monday patterns) and triggers pointless resets, so compare against seasonal baselines; you **cannot reproduce a prediction** because the model changed seconds later unless each prediction logs the snapshot id.

Cross-Connection Scenarios

- Retraining triggers and storms: Module 05 sections 1 and 9 (legacy).
- Bandits as online learners with exploration: 7.18.
- Drift taxonomy and monitors: Module 08 section 2 (legacy).
- Snapshot versioning in the registry: Module 04 (legacy).

Production Anti-Patterns

- Online learning with no snapshotting: a bad batch of labels corrupts the model and there is nothing to roll back to.
- Trusting delayed labels as immediate: the model learns from a biased early subset of outcomes (fast-resolving cases) and drifts.

- Auto-promoting every update: any bug, poisoned batch or feedback loop reaches users within minutes.

Minimal Working Code

Online logistic regression with SGD and prequential evaluation on a stream whose concept flips halfway, against a frozen model.

```
import numpy as np

rng = np.random.default_rng(0)
n, d, lr = 6000, 3, 0.05
w_true = np.array([2.0, -1.0, 1.0])
X = rng.normal(size=(n, d))
sign = np.where(np.arange(n) < n // 2, 1.0, -1.0)[:, None]     # concept flips at n/2
sigmoid = lambda z: 1 / (1 + np.exp(-z))
y = (rng.random(n) < sigmoid(sign[:, 0] * (X @ w_true))).astype(float)

w, online_ok = np.zeros(d), np.zeros(n)
for i in range(n):
    p = sigmoid(X[i] @ w)
    online_ok[i] = (p > 0.5) == y[i]            # predict first (prequential) ...
    w += lr * (y[i] - p) * X[i]                 # ... then update
wf = np.zeros(d)                                # frozen model: trained on the first 1000 only
for i in range(1000):
    wf += lr * (y[i] - sigmoid(X[i] @ wf)) * X[i]
frozen_ok = ((sigmoid(X @ wf) > 0.5) == y).astype(float)

for name, ok in [("online", online_ok), ("frozen", frozen_ok)]:
    print(f"{name}: before drift {ok[1000:3000].mean():.3f}, after drift {ok[4000:].mean():.3f}"
;)
```

A Page-Hinkley drift detector on an error stream, with an alarm hook that rolls back weights to the last snapshot preceding the estimated change point.

```
import random

class PageHinkley:
    def __init__(self, delta=0.05, threshold=15.0):
        self.delta, self.threshold = delta, threshold
        self.n, self.mean, self.cum, self.min_cum, self.change_at = 0, 0.0, 0.0, 0.0, 0
    def update(self, x):
        self.n += 1
        self.mean += (x - self.mean) / self.n
        self.cum += x - self.mean - self.delta
        if self.cum < self.min_cum:
            self.min_cum, self.change_at = self.cum, self.n
        return self.cum - self.min_cum > self.threshold      # True = drift alarm

rng = random.Random(0)
errors = [rng.random() < (0.10 if t < 500 else 0.40) for t in range(800)]
ph, weights, snapshots = PageHinkley(), [0.0], {0: [0.0]}   # snapshot 0 = initial weights
for t, err in enumerate(errors, start=1):
    weights[0] += 0.01                          # stand-in for a learning update
    if t % 100 == 0:
        snapshots[t] = list(weights)            # versioned snapshot every 100 steps
    if ph.update(float(err)):
        good = max(s for s in snapshots if s <= ph.change_at)
        weights = list(snapshots[good])         # rollback hook
        print(f"alarm at t={t}, change near {ph.change_at}, rolled back to snapshot {good}")
        ph = PageHinkley()                      # reset the detector
        break
```

Interview Angle

Q: When would you choose online learning over scheduled retraining, and what safeguards do you add? **A:** Choose it when the target changes faster than a retrain cycle, labels arrive quickly and the algorithm supports incremental updates, such as ad ranking or fraud. Otherwise scheduled retraining is more reproducible. Safeguards: shadow model, clipped updates, label quarantine, versioned snapshots, canary and rollback. Trade-off: adaptation speed versus auditability. Check: prequential accuracy against a frozen baseline and time to recover after an injected shift.

Q: How do you audit a prediction made by a continuously updating model? **A:** Log the snapshot id (and feature values) with every prediction, keep snapshots immutable for the retention period, and be able to reload the exact snapshot and replay the request. Trade-off: storage for snapshots and logs versus explainability. Check: a replay of a random sample of logged predictions reproduces scores within numerical tolerance.

14.5 Real-Time Inference Pipelines: Stateful Scoring, Backpressure and Dead Letters

Conceptual Deep Dive

A real-time pipeline has stages: **ingest**, **validate**, **enrich** from the online store, **score**, **act** and **log**. A **synchronous request path** (caller waits) suits user-facing decisions; an **asynchronous stream path** (consumer reads events, writes scores to a topic) suits decoupled actions such as alerts. Many systems use both: sync for the decision, stream for enrichment and monitoring.

Backpressure: when consumers cannot keep up, lag grows. **Consumer lag** is the primary health signal for streaming services, more informative than CPU. Autoscale on lag (a KEDA-style scaler is the typical Kubernetes mechanism; see Module 09 section 3 (legacy)), but remember that parallelism is capped by the number of partitions. Use bounded buffers inside consumers so memory does not grow without limit.

Failure handling: a **poison message** (unparseable or violating the schema) must not block the partition, so send it to a **dead-letter topic** with the error and original payload, and provide replay tooling after the fix. Manage schema evolution with a **schema registry** and compatibility rules; make consumers **idempotent** (dedup by event id) because delivery is at-least-once.

Ordering and partitioning: key by entity id so all events for an entity land on one partition and per-entity order is preserved, which stateful features and idempotent updates rely on. **Rebalances** (consumers joining or leaving) can redeliver uncommitted messages, so duplicates are normal.

Latency budget end to end includes broker delay, enrichment lookups, model compute and sink write; batching several events per model call raises throughput at a small, bounded delay cost.

Observability: lag, throughput, per-stage latency, DLQ rate, rebalance count and the score distribution (a sudden shift is a data problem). Alert on lag growth rate and DLQ age, not just absolute size, because a steady backlog and a growing one need different responses.

Edge cases: a **rebalance causes duplicate scores**, so downstream actions must be idempotent; a **schema change breaks all consumers at once** unless compatibility is enforced in CI; the **DLQ grows silently** unless its size and age are alerted on.

Cross-Connection Scenarios

- Streaming scorer example and exactly-once notes: Module 06 section 6 (legacy).
- Windowed features feeding the scorer: 14.2; freshness SLOs: 14.1.
- Load shedding and timeouts for the synchronous path: 6.17.
- Alerting on lag and DLQ rate: Module 08 section 5 (legacy).

Production Anti-Patterns

- Ack before the effect is durable: a crash loses the event permanently, and the score or action never happens.
- Dropping malformed messages without a record: schema bugs go unnoticed and nobody can replay the lost events.
- Single partition for all keys: throughput is capped by one consumer and per-entity ordering is unnecessary overhead.

Minimal Working Code

An at-least-once consumer simulation: idempotent sink keyed by event id, DLQ for unparseable messages and commit only after the write, with a crash between write and commit.

```
import json

def consume(log, state, sink, dlq, crash_at=None):
    """Process from the committed offset; return False if the worker 'crashes'."""
;"
    while state["committed"] < len(log):
        off = state["committed"]
        raw = log[off]
        try:
            evt = json.loads(raw)
            eid, score = evt["id"], round(evt["amount"] / 1000, 3)
        except (ValueError, KeyError, TypeError):
            dlq.append({"offset": off, "payload": raw})      # keep payload for repla
y
            state["committed"] = off + 1
            continue
        sink[eid] = score                                   # idempotent upsert by event id
        if crash_at == off and not state.get("crashed"):
            state["crashed"] = True
            return False                                    # crash after effect, before commit
        state["committed"] = off + 1
    return True

log = ['{"id": "a", "amount": 1500}', "not json", '{"id"
;: "b", "amount": 200}', '{"id": "c", "amount": 90}']
state, sink, dlq = {"committed": 0}, {}, []
print("first run finished:", consume(log, state, sink, dlq, crash_at=2))
print("restart finished:  ", consume(log, state, sink, dlq, crash_at=2))   # redelivers offset
2
assert sink == {"a": 1.5, "b": 0.2, "c": 0.09} and len(dlq) == 1
print(sink, "dlq:", dlq)
```

A lag-based autoscaling decision with a scale-down cooldown, capped by the partition count, run against a traffic burst.

```
import math

def desired_replicas(lag, arrival_rate, per_replica_rate, drain_s, partitions):
    need = arrival_rate + lag / drain_s                 # keep up and drain backlog within drain_s
    return max(1, min(partitions, math.ceil(need / per_replica_rate)))

def simulate(rates, per_replica=100.0, partitions=12, drain_s=60.0, cooldown=3, tick=10.0):
    lag, replicas, below, history = 0.0, 1, 0, []
    for t, rate in enumerate(rates):
        lag = max(0.0, lag + (rate - replicas * per_replica) * tick)
        want = desired_replicas(lag, rate, per_replica, drain_s, partitions)
        if want > replicas:
            replicas, below = want, 0                   # scale up immediately
        elif want < replicas:
            below += 1
            if below >= cooldown:                       # scale down only after sustained calm
                replicas, below = want, 0
        else:
            below = 0
        history.append((t, round(lag), replicas))
    return history

rates = [100] * 5 + [900] * 10 + [100] * 10
for t, lag, reps in simulate(rates)[::3]:
    print(f"tick {t:2d}: lag {lag:6d} replicas {reps}")
```

Interview Angle

Q: Design a fraud scoring stream with replay and DLQ. What are the failure modes? **A:** Topic keyed by card id, validated consumers, enrichment from the online store, scoring, then output topic plus prediction log. Commit offsets after the write, dedup by transaction id, send malformed events to a DLQ with payload and error, and keep raw events archived for replay. Failure

modes: consumer lag, hot keys, schema breaks, online-store timeouts, duplicate actions on rebalance. Trade-off: latency versus durability. Check: lag p99, DLQ rate and age, and a replay test reproducing scores.

Q: Why key by entity id and what breaks if you don't? **A:** Keying routes all events of one entity to one partition, preserving their order and letting a consumer hold that entity's state locally. Without it, events for an entity are processed concurrently on different consumers, so velocity features see stale or out-of-order state and idempotent updates race. Trade-off: hot entities create skew. Check: per-partition throughput variance and a test that per-entity event order is preserved at the sink.